

PARTIU MARROCOS / KEROMINDPARTIU MARROCOS / KEROMIND

Partiu Marrocos — Status Completo do Projeto

Relatório

15 de setembro de 2026

Levantamento completo, não resumido, de tudo que já foi construído e tudo que falta — cobrindo os dois repositórios que existem hoje: o **CRM real** (`packages/db` + `apps/web`, este monorepo) e o **site público** estático (`site-original/partiumarrocos.com.br`).

Data do levantamento: 15/09/2026. Métricas verificadas nesta data por execução real (não por memória): **468 testes automatizados passando** (364 em `packages/db` + 104 em `apps/web`), **39 migrations aplicadas** (100% aditivas, nenhuma destrutiva), typecheck e build de produção limpos (21 rotas em `apps/web`).

PARTE 1 — FUNDAÇÃO DE PLATAFORMA

Tudo nesta parte é infraestrutura genérica, reaproveitável por qualquer domínio novo que venha a ser construído — não é "feature de turismo", é a base que torna o resto seguro.

PM-BLOQ-001 — Secret Provider

Status: concluído.

Eliminou `Tenant.aiApiKey` em texto plano, substituindo por uma abstração `SecretProvider` (`packages/db/src/secret-provider.ts`). Toda credencial nova do sistema (WhatsApp, futuros gateways, etc.) passa por essa camada — nunca fica hardcoded, nunca é logada, nunca é retornada em API, nunca é persistida em texto plano em nenhuma tabela.

T1 — Gates/Approval + Audit Log

Status: concluído.

- **Gates** (`packages/db/src/gates.ts`): mecanismo de aprovação humana obrigatória para ações sensíveis (descontos fora de política, estornos, pagamento de comissão, etc.). Máquina de estados própria, com expiração por sweep preguiçoso (sem cron).
- **Audit Log** (`packages/db/src/audit.ts`): log de auditoria **append-only** — uma vez gravado, um evento não pode ser alterado nem apagado por caminho normal (só via `rls_bypass()` em cascade de exclusão de Tenant, documentado como exceção estrutural). Toda ação sensível de todo domínio construído depois passa por aqui.

T2 — Cost Control

Status: concluído.

Camada genérica de custo técnico/IA: medição → registro → acumulação → limite → alerta → bloqueio → Gate. Existe para impedir que uma automação (ex.: agente de IA, campanha) gaste sem controle — mede antes de gastar, não só relata depois.

T3 — Tool Broker mínimo + Yalla Camadas 1 e 2

Status: concluído.

- **Tool Broker** (`packages/db/src/tools/broker.ts` + `registry.ts` + `grants.ts`): registro **default-deny** de ferramentas que um agente de IA pode executar — nada é permitido a menos que explicitamente registrado, com schema de input/output, classificação de risco, autorização, execução com timeout real (via `AbortSignal`) e auditoria própria.
- **Definições de ferramenta já registradas**: atendimento, contatos, conversas, leads, notas, repescagem, tarefas (`packages/db/src/tools/definitions/*.ts`) — o conjunto de ações que o agente Yalla pode efetivamente chamar hoje.

- **Yalla Camadas 1 e 2:** camadas de raciocínio/consulta do agente — explicitamente **sem Camada 3** (nenhuma ação financeira/comercial crítica ou irreversível é executada pelo agente sozinho; isso sempre passa por Gate).

T5 + T5-FIX — Job/Execution Engine

Status: concluído, com uma rodada extra de hardening.

Fila de execução assíncrona real (`packages/db/src/jobs/engine.ts`): claim atômico via **SKIP LOCKED** (sem duplo processamento sob concorrência real, testado com carga de 100 jobs/múltiplos workers), retry com backoff, dead-letter (com retry manual via RBAC), lease/heartbeat/recuperação de worker travado, dependência entre jobs (A→B), integração com Gate (job pode pausar esperando aprovação humana e retomar sozinho), histórico de execução imutável depois de terminal (**UPDATE/DELETE** via SQL bruto numa Execution já **SUCCEEDED** lança exceção, testado).

T5-FIX foi uma rodada curta de correção antes de declarar T5 definitivamente concluído — corrigiu timeout real via **AbortSignal** (nenhuma promise órfã gera unhandled rejection quando um handler é abortado).

Hoje 2 tipos de job reais rodam nesse motor: `whatsapp.enviar_mensagem` e `travel_document.verificar_pendencias`, além de `lead.repescar_elegibilidade`.

PARTE 2 — CRM COMERCIAL

PM-SANEAMENTO-01

Status: concluído. Rodada de correção/consolidação sobre a base existente antes de evoluir o CRM (detalhes em `docs/PM_SANEAMENTO_01_FECHAMENTO.md`).

PM-CRM-FIN-ARCH-01 — Convergência CRM + Finance Core Internacional

Status: concluído (arquitetura/contratos).

Definiu os contratos base pra internacionalização financeira sem implementar regra fiscal nenhuma ainda: **Market** (código, idioma padrão, idiomas permitidos, moeda padrão, moedas permitidas, timezone — **sem** campos de conteúdo/SEO/pagamento/canais, que ficariam pra F2/F4) e **Jurisdiction** (vazio de propósito — nenhuma instância real, nenhuma regra fiscal). Verificado: **zero** ocorrências de `if idioma === "pt-PT"` ou equivalente hardcoded em qualquer lugar do código dessa rodada.

CRM Evolution 01

Status: concluído — 11 itens do escopo autorizado entregues (detalhe completo em `docs/CRM_EVOLUTION_01_FECHAMENTO.md`). Evoluiu o CRM base: Leads/Pipeline/Contatos/Conversas com funil configurável por tenant.

Lead Scoring + Next Best Action

Status: concluído (`packages/db/src/crm/lead-scoring.ts`, `next-best-action.ts`).

Scoring determinístico (não é IA generativa) — soma pontos por sinais objetivos (telefone/email informado, campos de preferência preenchidos, etapa do funil, recência de atividade, origem rastreada). Next Best Action é **recomendação**, nunca execução automática — sempre precisa de um humano clicar.

Repescagem estruturada de leads

Status: concluído (`packages/db/src/tools/definitions/repescagem.ts` + job `lead.repescar_elegibilidade`).

Reavalia elegibilidade **na hora de rodar** (nunca confia em uma condição congelada no momento do agendamento) — nunca envia WhatsApp sem elegibilidade real confirmada nesse instante.

Proposal Foundation 01

Status: concluído (`packages/db/src/proposals.ts`, `packages/db/src/crm/proposta-politica.ts`).

Proposta comercial real e **versionada** — rascunho → enviada → aceita/recusada. Versão anterior nunca é reescrita; uma nova edição gera nova versão. Avaliação de política comercial embutida no fluxo de envio.

Finance Core Foundation 02

Status: concluído (fundação em contrato TS puro, sem tabela ainda nessa rodada).

Definiu 9 entidades financeiras como contratos TypeScript (`packages/db/src/finance/types.ts`) sem promover nenhuma a tabela real ainda — decisão deliberada de "contrato primeiro, tabela só quando houver consumidor real". Ver Parte 3 pra saber quais dessas 9 foram efetivamente promovidas depois.

T6 — Attribution

Status: concluído.

Captura de atribuição de marketing (`packages/db/src/attribution.ts`): UTM (source/medium/campaign/content/term), `gclid`, `fbclid`, landing page, referrer — gravados como **AttributionTouch** na mesma transação que cria o Contact/Lead (nunca lead sem atribuição por falha no meio do caminho, nunca atribuição órfã). Consulta analítica de leads por atribuição já implementada.

F2 — Internacionalização (i18n) Foundation

Status: concluído, com escopo estritamente limitado — e essa limitação é deliberada, não uma lacuna esquecida.

`packages/db/src/i18n/` fornece: catálogo **LOCALES** com **BR** (pt-BR/BRL/America/Sao_Paulo) e **PT** (pt-PT/EUR/Europe/Lisbon); `resolverLocale(marketCode)` — nunca adivinha por idioma do navegador/IP, só resolve por código explícito; formatação de moeda e data/hora corretas por mercado (moeda pode ser diferente do locale, timezone explícito prova horas diferentes pro mesmo instante).

O que F2 explicitamente NÃO faz — declarado no próprio relatório de fechamento: `"nunca tradução automática de texto, só o par idioma/moeda/fuso associado a um mercado"`. Isso é relevante porque foi exatamente sobre isso que você perguntou nesta conversa — ver Parte 4.

PARTE 3 — OPERAÇÃO DE VENDA (PM-NIGHT-RUN-02, em andamento)

Essa é a frente mais recente, ainda em execução sob autorização aberta. 5 de 8 blocos concluídos até agora, todos GREEN (nenhum bloqueio, nenhuma decisão de fundador pendente em nenhum deles).

Etapas 1 — Booking Foundation

Status: concluído.

Booking (`packages/db/src/booking.ts`) — reserva comercial real, **distinta** de Proposal (a Proposal ACEITA vira snapshot imutável; o Booking nunca duplica esse dado, só referencia). Idempotência estrutural: uma Proposal só gera um Booking (`@@unique`), garantido pelo banco, não só por lógica de aplicação. Máquina de estados: **AGUARDANDO_PAGAMENTO** → **PAGAMENTO_PARCIAL/PAGO** →

AGUARDANDO_DOCUMENTOS/CONFIRMADA → EM_OPERACAO → CONCLUIDA, com CANCELADA de qualquer estado não-terminal exceto EM_OPERACAO (viagem em andamento não cancela sozinha). **Traveler** (passageiro) promovido a tabela real, privacy-by-design desde o início (nenhum documento sensível nesta etapa).

Etapa 2 — Payment Foundation

Status: concluído.

Payment (`packages/db/src/payment.ts`) — domínio **provider-neutro** (`PAYMENT ≠ GATEWAY`, declarado explicitamente). **Nenhum gateway real conectado** — modo manual/offline, **provider/providerReference** sempre `null` hoje. Suporta pagamento único, sinal+saldo, parcelamento (N linhas de Payment por Booking, sem campo extra). Sincroniza o status do Booking automaticamente conforme pagamentos entram, mas nunca regride um Booking já avançado. **Estorno sempre via Gate financeiro real**, com idempotência genuína (reconfirmar o mesmo Gate aprovado nunca duplica o valor estornado — bug real encontrado e corrigido nesta etapa, antes de qualquer teste pegar). **idempotencyKey** já existe no schema, pronta pra um futuro webhook de gateway real, sem consumidor ainda.

Etapa 3 — Finance Core Real 01

Status: concluído.

Reavaliação das 9 entidades financeiras da Finance Core Foundation 02 agora que Booking/Payment são consumidores reais:

- **`Commission` promovida a tabela real** — **Booking.responsavelId** é consumidor genuíno. Máquina de estados **PREVISTA → CONFIRMADA → PAGA** (CANCELADA de Prevista/Confirmada). Pagamento sempre via Gate financeiro, com a mesma idempotência já validada em Payment.
- **`Receivable` NÃO virou tabela** — virou consulta computada sobre **Payment** (**listarContasARceber**), deliberadamente, pra não duplicar o mesmo dado que **Payment** PENDENTE/PROCESSANDO/PARCIALMENTE_PAGO já representa (dois lugares guardando a mesma verdade é risco de divergência).
- **`Refund` confirmado já resolvido** por **Payment.status** desde a Etapa 2 — removido do arquivo de contratos.
- As outras 6 (**FinancialAccount/FinancialCategory/CostCenter/RevenueCenter/FinancialTransaction/Payable**) continuam só contrato — sem consumidor real ainda, não promovidas.
- **`CommercialPolicy`** — tabela nova (1 linha/tenant) que corrige a limitação anterior de limiares de desconto fixos e hardcoded (15%/20%/10%). Agora configurável por tenant, com os mesmos valores como default (nenhuma mudança silenciosa de comportamento pra quem não configura). Verificado ponta a ponta em navegador real: um limiar customizado (8%) realmente muda o comportamento (uma proposta com 10% de desconto passou a exigir Gate, o que não aconteceria com o padrão antigo de 15%).

Etapa 4 — Travel Document Foundation

Status: concluído.

Separação real entre **REQUISITO** (**DocumentRequirement** — catálogo reusável por tenant, ex. "Passaporte válido") e **DOCUMENTO ENVIADO** (**TravelerDocument** — uma linha por passageiro × requisito). Desativar um requisito preserva o histórico já gerado. **adicionarTraveler** sincroniza automaticamente uma linha PENDENTE por requisito ativo (idempotente). 6 estados (**PENDENTE/ENVIADO/EM_ANALISE/APROVADO/REJEITADO/EXPIRADO**), máquina de estados testada. Estado agregado do Booking (**statusDocumentalDoBooking**) é **puramente informativo** — nunca auto-confirma o Booking sozinho.

Privacy-by-design real, não só declarada: `TravelerDocument` não tem nenhum campo de arquivo, URL ou caminho no schema — estruturalmente impossível gravar um arquivo hoje, não é uma regra de código que alguém possa esquecer de aplicar. Upload real fica pra quando houver storage seguro disponível (privado, autenticado, validação de MIME/tamanho, nomes aleatórios, sem SVG).

Prazos/pendências via Job Engine (`travel_document.verificar_pendencias`) — sinaliza internamente via **Note**, nunca envia WhatsApp sozinho (testado com asserção negativa explícita em todo cenário).

Etapa 5 — Trip Operation Foundation

Status: concluído (a mais recente).

Separação real entre **Booking** (reserva comercial) e **Trip** (execução operacional da viagem — roteiro, mercado, datas, timezone, responsável operacional, status operacional próprio). Nenhum campo comercial duplicado em Trip.

Decisão arquitetural central: relação **Trip ↔ Booking** é 1:N, nunca presumida 1:1 — `Booking.tripId` é FK nullable simples (Trip é o lado "1"). **Provado com teste dedicado:** dois Bookings diferentes do mesmo lead vinculados à mesma Trip sem conflito nenhum (cenário real de turismo em grupo, várias famílias/reservas na mesma execução operacional).

Máquina de estados: **PLANEJAMENTO → CONFIRMADA → EM_ANDAMENTO → CONCLUIDA, CANCELADA** só de Planejamento/Confirmada (mesma regra de não-cancelamento em andamento já usada em Booking).

Itinerário dia a dia (`TripItineraryDay`, numeração única por Trip) com atividades (`TripActivity` — nome, local, horário como texto livre, instruções, `visivelParaViajante` — já preparado pra Área do Viajante futura) e checklist operacional por categoria (`TripChecklistItem` — 8 categorias: Documentos/Pagamento/Fornecedores/Transporte/Hospedagem/Atividades/Transfer/Outro).

Motor genuinamente agnóstico de destino — nada no código assume Marrocos ou qualquer fornecedor específico; todo conteúdo real é dado inserido pelo usuário.

Verificado ponta a ponta em navegador real nesta sessão: Trip criada → dia de itinerário → atividade visível + atividade interna (rótulo "interno" confirmado) → item de checklist criado e marcado concluído → status movido pra Confirmada → fluxo completo Lead→Proposta→Booking→vínculo com Trip, com o contador de "Reservas vinculadas" atualizando corretamente.

PARTE 4 — COMUNICAÇÃO (WhatsApp, Inbox, Tradução)

Esta parte é a resposta direta ao que você perguntou nesta conversa — levantada com leitura de código nesta sessão, não por memória.

WhatsApp — envio real, funcional

- `apps/web/src/lib/whatsapp/cloud-api.ts` — chama a Meta Cloud API de verdade (não é mock).
- `apps/web/src/app/actions/whatsapp.ts` — `salvarContaWhatsapp()` (configuração da conta por tenant, credenciais via SecretProvider) e `responderWhatsapp()` (resposta manual do atendente).
- `apps/web/src/lib/jobs/definitions/whatsapp-enviar-mensagem.ts` — tipo de job real no Job Engine (T5): envia via Cloud API com retry/backoff, detecta a restrição da janela de 24h da Meta, documenta honestamente uma ressalva de duplicação at-least-once sob certos cenários de retry.
- `apps/web/src/app/api/webhooks/whatsapp/route.ts` — webhook de recebimento existe (lado de entrada de mensagem).

Inbox (chat) — existe, é funcional mas simples

`apps/web/src/app/(app)/inbox/page.tsx` — lista conversas de qualquer canal (filtradas por tenant), mostra o histórico de mensagens da conversa selecionada, mostra formulário de resposta quando o canal é WhatsApp e o usuário tem `atendimento.manage`. Mostra o conteúdo da mensagem **exatamente como veio** — sem nenhuma etapa de processamento.

Tradução automática de mensagem — NÃO EXISTE

Confirmado por busca de código em toda a base (`apps/web/src` + `packages/db/src`), zero ocorrências de qualquer termo relacionado a serviço de tradução (`translate`, `DeepL`, Google/Azure Translate, etc.). Nenhum campo no modelo `Message` guarda uma variante traduzida. O F2 (i18n) — que poderia parecer relacionado pelo nome — **não traduz texto**, é só formatação de moeda/data/fuso por mercado (ver Parte 2).

Isso corresponde exatamente ao que o documento de fundação do projeto já previa como roadmap de 5 fases (`DOCUMENTO-DE-FUNDACAO-PARTIU-MARROCOS.md`):

Fase	Conteúdo	Status hoje
1 — Lançamento comercial	CRM, WhatsApp, tracking, SEO base	base pronta
2 — Portugal	i18n, moeda, fuso	pronto (F2)
3 — Atendimento internacional	Tradução texto/áudio, ElevenLabs	não iniciado — "construir do zero"
4 — Marketing KeroMind	Ads, comentários, atribuição	parcial (T6 Attribution pronto; conectores de Ads não)
5 — Command Center	Briefing, scoring, NBA	parcial (Lead Scoring/NBA prontos; Command Center completo não)

A Fase 3 (tradução + voz) está **explicitamente proibida de começar** na autorização em andamento (PM-NIGHT-RUN-02 lista "F3 Voz/ElevenLabs" entre os blocos que nunca devem ser iniciados sem autorização própria).

PARTE 5 — SITE PÚBLICO (`partiumarroc.com.br`)

O que está pronto

Site estático completo em linguagem cinematográfica (GSAP 3 + ScrollTrigger): cold open com contagem regressiva, letterbox com timecode 24fps, hero com título animado + parallax, trailer com texto palavra a palavra, 7 motivos, filmstrip horizontal com 8 cenários clicáveis, fichas técnicas de 11 destinos (modal navegável), rota SVG que se desenha no scroll com contador de km, storyboard dia a dia de 3 roteiros (5/8/12 dias), pôsteres de preço, 9 experiências, 8 pratos + marquee, tour panorâmico arrastável (6 cenas), seção de vídeo, 9 curiosidades, depoimentos, elenco/equipe, FAQ (8 perguntas), cultura Amazigh com contadores, formulário de orçamento completo, créditos finais, chatbot "Yalla" decorativo (15 temas fixos, **não é o agente real do CRM**).

Acessibilidade: `prefers-reduced-motion` respeitado, navegação por teclado em modais.

Painel administrativo próprio (**admin.html**) com autenticação server-side, rate limit de tentativas, upload de imagem validado (sem SVG — vetor conhecido de XSS armazenado).

Integração real com o CRM: o formulário de orçamento grava o lead de verdade no CRM (**POST /api/public/leads**) antes de abrir o WhatsApp — testado ponta a ponta nesta sessão, com **apiBase** apontado pro CRM local (**http://localhost:3000**) e lead de teste confirmado gravado e depois removido.

O que falta — tudo conteúdo real, nada de código

Item	Onde	Status
WhatsApp real	js/data.js	placeholder 55999999999999
Nomes da equipe	index.html , seção "O Elenco"	placeholders
Instagram/Facebook	créditos finais	href="#"
Vídeo teaser	seção "Cena 10"	sem link real
Textos factuais (números, ★★★★★, afirmações geográficas)	várias seções	não comprovados, revisar antes de publicar
Senha do painel admin em produção	config.php no servidor	precisa ser configurada no cPanel real
PUBLIC_SITE_ORIGIN/ apiBase de produção	.env do CRM / js/data.js	hoje só aponta pro localhost de teste

PARTE 6 — O QUE FALTA, NO GERAL

Autorizado, ainda não construído (sequência em andamento)

1. **Traveler Area V1** — área do cliente final fora do CRM, pra consultar a própria viagem (itinerário, status, pendências de documento) — autenticação obrigatoriamente segura (link mágico com token expirável, nunca um ID previsível ou dado pessoal sozinho). Testes negativos de segurança obrigatórios (cliente A não vê Booking de B, token expirado falha, etc.).
2. **Notifications Foundation** — domínio genérico de notificação (IN_APP/EMAIL/WHATSAPP/PUSH/WEBHOOK) pros eventos que já existem (proposta enviada, pagamento, vencimento, documento pendente, viagem confirmada). Async via Job Engine, preservando idempotência/retry/auditoria.
3. **Post-Trip Foundation** — pós-viagem: follow-up, avaliação/NPS, depoimento **com consentimento explícito** (nunca publicação automática), nova oportunidade/indicação preservando o histórico do cliente.

Depois desses três, a autorização atual (PM-NIGHT-RUN-02) determina **parada obrigatória** — não é esquecimento, é limite deliberado pra reavaliação sua antes de continuar.

Explicitamente fora de escopo até nova autorização sua

- **T4** — Router multi-modelo de IA
- **F3** — Tradução de texto/áudio + Voz (ElevenLabs) — a pergunta que você fez nesta conversa
- **F4** — Conectores de Marketing (Ads, campanhas)

- **F5 — Command Center completo**
- **F6 — Country Packs fiscais** (Portugal, Brasil) — nenhuma regra fiscal concreta existe ainda, de propósito
- **ERP completo / Supplier Engine completo**
- **Gateway de pagamento de produção** — precisa de autorização e credencial explícitas seus
- **Emissão fiscal**
- **App mobile nativo**

Limitações técnicas já conhecidas (declaradas, não escondidas)

- Upload real de documento de viagem não existe (falta storage seguro).
- Nenhum gateway de pagamento real conectado (modo manual/offline).
- WhatsApp Business (WABA) não homologado em conta real de produção — a integração técnica existe, mas nunca foi validada com número real.
- Nenhuma tradução automática de mensagem (Fase 3, não iniciada).

Decisões que só você pode tomar

- Se e quando iniciar a Fase 3 (tradução/voz).
- Se e quando conectar um gateway de pagamento real (e qual).
- Inicializar o repositório Git deste projeto (**git init**) — decisão de infraestrutura, não técnica, ainda pendente.
- Dados reais do site público (WhatsApp, nomes da equipe, redes sociais, vídeo, textos factuais revisados).
- O que priorizar depois da parada obrigatória em Post-Trip Foundation: Traveler Area mais robusta, Notifications mais completo, ou começar uma das frentes hoje fora de escopo.